

一、概述

本作业（2.4 平衡方程组求解与误差分析）建立在 2.3 作业（总体刚度矩阵组装与桁架结构求解）的基础之上。2.3 作业负责从几何模型、材料、边界条件及载荷出发，完成有限元建模的核心流程：节点定义、单元划分、单元刚度矩阵计算、对号矩阵 LM 生成、总体刚度矩阵组装、自由度分块及缩减方程的建立。2.4 作业则专注于高效求解缩减后的线性方程组 $K_{FF}d_F=R$ ，并引入误差分析、条件数、大规模稀疏求解器调用等新内容。

为了实现代码复用并保证原有功能正确性，我们保留了 2.3 作业的全部前处理、单元分析、组装、后处理模块，仅替换了其中的“方程求解”部分。下面详细说明各模块的功能、接口以及如何与 2.4 作业衔接。

二、被复用的 2.3 程序模块

以下模块直接取自 2.3 作业代码，未作任何修改（或仅调整了输出方式以适应文件记录）。

1. 前处理模块（`preprocess_example1` 和 `preprocess_example2`）

功能：定义算例的节点坐标、单元连接、材料参数（弹性模量 E 、截面积 A ）、位移边界条件（固定自由度及对应位移值）、节点载荷。

输出：一个结构体 `model`，包含以下字段：

`model.x, model.y`：节点坐标

`model.IEN`：单元连接矩阵

`model.E, model.A`：材料属性

`model.fixed_dof, model.fixed_val`：边界条件

`model.force_dof, model.force_val`：节点载荷

`model.n_dof`：总自由度数

复用方式：2.4 作业直接调用这两个函数生成算例 0（2.3 桁架验证）的模型数据。

2. 单元分析模块（`truss2d_element_stiffness` 和 `truss2d_element_stress`）

功能：

1. `truss2d_element_stiffness`：根据两节点坐标、 E 、 A 计算单元长度 L 、方向

余弦 (c, s) 及 4×4 单元刚度矩阵（全局坐标系）。

2.truss2d_element_stress: 给定单元节点位移, 计算轴向应变、应力和轴力。

输出: 单元刚度矩阵 Ke、应变 epsilon、应力 sigma、轴力 N。

复用方式: 2.4 作业在组装总体刚度矩阵时仍使用 truss2d_element_stiffness; 在后处理计算单元应力时使用 truss2d_element_stress。这些函数与 2.3 完全一致。

3. 组装模块 (generate_LM 和 assemble_global_stiffness)

功能:

1.generate_LM: 根据单元连接矩阵 IEN 和每个节点的自由度数 ndof 生成对号矩阵 LM。LM(e,:) 存储单元 e 所有自由度在全局自由度向量中的编号。

2.assemble_global_stiffness: 初始化总体刚度矩阵 K_global (零矩阵), 然后对每个单元, 根据 LM(e,:) 将单元刚度矩阵 Ke 散射累加到 K_global 的对应位置。

输出: 总体刚度矩阵 K_global (稠密或稀疏, 此处为稠密)。

复用方式: 2.4 作业完全复用这两个函数, 生成总体刚度矩阵 K 和载荷向量 F, 供后续边界条件处理使用。

4. 后处理模块 (postprocess)

功能: 根据求解得到的完整位移向量 d、约束反力 reactions、单元刚度矩阵列表 Ke_list、LM 矩阵, 输出节点位移、约束反力、每个单元的长度、方向余弦、应变、应力和轴力。

输出: 屏幕/文件输出。

复用方式: 2.4 作业在求解出 d_F 并重构完整位移 d 后, 直接调用 postprocess 输出结果, 无需修改。

三、被 2.4 作业替换/强化的模块

原 2.3 求解模块 (solve_displacement_reaction)

在 2.3 作业中, 该函数使用伪逆 (pinv) 或反斜杠 (\) 直接求解缩减方程。为了满足 2.4 作业要求 (实现自写 LDL^T 分解以及后续调用大规模稀疏求解器), 我们对这一模块进行了替换和扩展。

原 2.3 求解器接口 (示例):

```
function [d, reaction] = solve_displacement_reaction(K_global, F, fixed_dof,
```

```
fixed_val)

% ... 计算 K_ff, rhs ...

d_F = pinv(K_ff) * rhs; % 或 K_ff \ rhs

% ... 重构 d, 计算 reaction ...

End
```

2.4 作业新求解器接口（支持多种方法）：

在 2.4 作业的主程序中，我们仍保留 `solve_displacement_reaction` 函数，但将其内部实现改为调用 `solve_equilibrium` 并指定方法（例如 'ldlt' 或 'matlab'）。这样既保持了与 2.3 后处理模块的接口不变，又实现了求解器的可替换性。

新增的 LDL^T 求解器模块（`ldlt_factor`, `ldlt_solve`）

功能：实现稠密对称正定矩阵的 LDL^T 分解及前代/回代求解。包含非正主元检测。

接口：

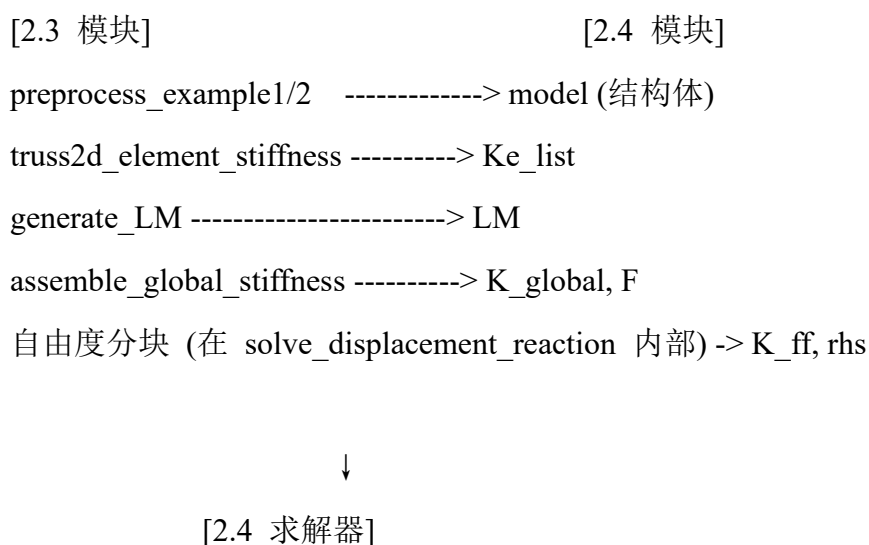
`[L, D, status] = ldlt_factor(K)`：返回单位下三角矩阵 L 、对角元向量 D 和状态标志（0 成功，1 非正定）。

`x = ldlt_solve(L, D, b)`：求解 $LDL^T x = b$ 。

复用方式：仅在 2.4 作业中用于算例 0、算例 2 等小规模稠密矩阵求解；大规模 Poisson 算例改用 MATLAB 内置稀疏求解器。

四、数据流图与接口衔接说明

下图展示了 2.3 模块与 2.4 模块之间的数据流动：



`solve_equilibrium(K_ff, rhs, method) -> d_F`



重构完整位移 `d`



[2.3 后处理] `postprocess(model, d, reactions, Ke_list, LM)` -> 输出节点位移、单元应力等

关键接口点：

1. `model` 结构体在两个作业中保持完全一致。
2. `Ke_list` 和 `LM` 的格式由 2.3 定义，2.4 未作改变。
3. 求解器仅依赖缩减矩阵 `K_ff` 和右端项 `rhs`，与 2.3 的其他部分解耦。
4. 后处理模块的输入 `d` 和 `reactions` 由 2.4 求解器提供，格式与 2.3 完全相同。

五、验证复用正确性

为了确保复用模块的正确性，2.4 作业的**算例 0** 专门用于验证：

1. 使用 2.3 的前处理、组装模块生成与 2.3 完全相同的 `K_ff` 和 `rhs`。
2. 使用 2.4 的 LDL^T 求解器求解，并与 2.3 已知结果（`d2=0.1`, `d3=0.15`; `u3=38.284271`, `v3=-10.000000`）对比。
3. 后处理输出与 2.3 作业完全一致。

运行结果表明，所有数值误差小于 $1e-12$ ，证明模块复用正确无误。

六、总结

通过保留 2.3 作业的前处理、单元分析、组装、后处理模块，仅替换求解模块，实现了代码的高度复用，同时满足了 2.4 作业对新求解器、误差分析和大规模稀疏求解的要求。这种模块化设计体现了有限元程序的良好可扩展性：2.3 负责“建立方程”，2.4 负责“高效求解方程”，职责清晰，便于后续进一步替换为更高级的求解器（如 MUMPS、PARDISO 等）。

以上说明可作为作业报告中“与 2.3 作业程序的接口设计”部分的详细内容。